



UASLP
Universidad Autónoma
de San Luis Potosí

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 14

Sistema de Cruce Peatonal Síncrono

Implementación con Raspberry Pi Pico W

Alumno: César Gael Mancilla Estrada

Clave: 382031

Carrera: Ingeniería en Mecatrónica

Materia: Microcontroladores

Profesor: Ing. Jesús Padrón

Fecha de entrega: 24 de abril de 2026

«La automatización no elimina puestos de trabajo, los transforma»

Índice

1. Introducción	2
1.1. Contexto y Justificación	2
1.2. Importancia en la Industria	2
1.3. Objetivos	2
2. Marco Teórico	2
2.1. La Raspberry Pi Pico W	2
2.2. Máquinas de Estados Finitos (FSM)	3
2.3. Multiplexación de Displays	3
3. Desarrollo	3
3.1. Material Utilizado	3
3.2. Diagrama de Conexiones	4
3.2.1. Conexión de Displays 7 Segmentos	4
3.2.2. Conexión de Semáforos	4
3.2.3. Conexión de Botones y Buzzer	4
3.3. Diagrama Esquemático del Circuito	5
3.4. Maqueta Física	6
3.5. Diagrama de Flujo de la Máquina de Estados	6
4. Código Fuente	7
4.1. Archivo: semaforo.h	7
4.2. Archivo: Practica_14.c (Principal)	8
5. Resultados y Pruebas	9
5.1. Funcionamiento del Sistema	9
5.2. Prueba de Funcionamiento	10
5.3. Dificultades y Soluciones	10
6. Conclusiones	11
6.1. Posibles Mejoras	11
7. Referencias	11

1. Introducción

1.1. Contexto y Justificación

Los sistemas de control de tráfico vehicular y peatonal son elementos críticos en la infraestructura urbana moderna. Un cruce peatonal mal sincronizado puede generar accidentes, congestión vehicular y poner en riesgo la vida de los transeúntes. La implementación de sistemas síncronos de control permite una gestión eficiente del flujo vehicular y peatonal, reduciendo tiempos de espera y maximizando la seguridad.

La presente práctica aborda el diseño e implementación de un sistema de cruce peatonal utilizando un microcontrolador **Raspberry Pi Pico W**, programado en lenguaje C con el SDK oficial de Raspberry Pi. El sistema coordina dos semáforos vehiculares (rojo, amarillo y verde) y dos semáforos peatonales (rojo y verde), además de incorporar contadores regresivos con displays de 7 segmentos para indicar al peatón el tiempo restante para cruzar.

1.2. Importancia en la Industria

Los sistemas de control de tráfico basados en microcontroladores son ampliamente utilizados en la industria de la automatización urbana. El conocimiento adquirido en esta práctica sienta las bases para comprender y desarrollar sistemas más complejos de control de tráfico inteligente.

1.3. Objetivos

- Entender el funcionamiento de un sistema síncrono aplicado a un cruce peatonal
- Aplicar el uso de LEDs, displays de 7 segmentos, botones y lógica de control coordinada
- Implementar una máquina de estados finitos para la gestión de secuencias de semáforos
- Diseñar y construir una maqueta funcional que represente un cruce peatonal real

2. Marco Teórico

2.1. La Raspberry Pi Pico W

La **Raspberry Pi Pico W** es una placa de desarrollo basada en el microcontrolador **RP2040**, diseñado por Raspberry Pi Foundation. Sus características principales incluyen:

- Procesador ARM Cortex-M0+ de doble núcleo a 133 MHz
- 264 KB de SRAM y 2 MB de memoria Flash
- 26 pines GPIO multifunción
- Conectividad Wi-Fi 802.11n (2.4 GHz)

2.2. Máquinas de Estados Finitos (FSM)

Una **máquina de estados finitos** es un modelo matemático de computación utilizado para diseñar sistemas secuenciales. En este proyecto, la FSM gestiona cinco estados principales como se muestra en el diagrama de la Sección 3.4.

2.3. Multiplexación de Displays

La **multiplexación** es una técnica que permite controlar múltiples displays de 7 segmentos utilizando un número reducido de pines GPIO, alternando rápidamente entre ellos para aprovechar la persistencia visual del ojo humano.

3. Desarrollo

3.1. Material Utilizado

Tabla 1: Lista de materiales

Componente	Cantidad	Especificación
Raspberry Pi Pico W	1	Microcontrolador RP2040
LEDs Rojos 5mm	4	Semáforos
LEDs Amarillos 5mm	2	Semáforos vehiculares
LEDs Verdes 5mm	4	Semáforos
Displays 7 segmentos	4	Cátodo común
Resistencias 1k Ω	7	Para segmentos
Resistencias 330 Ω	10	Para LEDs
Botones pulsadores	2	Solicitud de cruce
Buzzer pasivo	1	Señal acústica
Protoboard	1	Montaje
Cables jumper	30+	Conexiones

3.2. Diagrama de Conexiones

3.2.1. Conexión de Displays 7 Segmentos

Tabla 2: Conexiones de displays 7 segmentos

Componente	GPIO	Conexión
Segmento A	GPIO 0	Pin 1 $\rightarrow$ [1k Ω] $\rightarrow$ Ánodo A
Segmento B	GPIO 1	Pin 2 $\rightarrow$ [1k Ω] $\rightarrow$ Ánodo B
Segmento C	GPIO 2	Pin 4 $\rightarrow$ [1k Ω] $\rightarrow$ Ánodo C
Segmento D	GPIO 3	Pin 5 $\rightarrow$ [1k Ω] $\rightarrow$ Ánodo D
Segmento E	GPIO 4	Pin 6 $\rightarrow$ [1k Ω] $\rightarrow$ Ánodo E
Segmento F	GPIO 5	Pin 7 $\rightarrow$ [1k Ω] $\rightarrow$ Ánodo F
Segmento G	GPIO 6	Pin 9 $\rightarrow$ [1k Ω] $\rightarrow$ Ánodo G
Disp1 Decenas	GPIO 8	Pin 11 $\rightarrow$ Cátodo común
Disp1 Unidades	GPIO 9	Pin 12 $\rightarrow$ Cátodo común
Disp2 Decenas	GPIO 7	Pin 10 $\rightarrow$ Cátodo común
Disp2 Unidades	GPIO 28	Pin 34 $\rightarrow$ Cátodo común

3.2.2. Conexión de Semáforos

Tabla 3: Conexiones de semáforos

LED	GPIO	Pin Físico	Color
V1 Rojo	GPIO 15	Pin 20	Rojo
V1 Amarillo	GPIO 14	Pin 19	Amarillo
V1 Verde	GPIO 13	Pin 17	Verde
V2 Rojo	GPIO 12	Pin 16	Rojo
V2 Amarillo	GPIO 11	Pin 15	Amarillo
V2 Verde	GPIO 10	Pin 14	Verde
P1 Rojo	GPIO 16	Pin 21	Rojo
P1 Verde	GPIO 17	Pin 22	Verde
P2 Rojo	GPIO 18	Pin 24	Rojo
P2 Verde	GPIO 19	Pin 25	Verde

3.2.3. Conexión de Botones y Buzzer

Tabla 4: Conexiones de botones y buzzer

Componente	GPIO	Pin Físico	Conexión
Botón 1 (Peatón 1)	GPIO 20	Pin 26	GPIO $\leftrightarrow$ Botón $\leftrightarrow$ GND
Botón 2 (Peatón 2)	GPIO 21	Pin 27	GPIO $\leftrightarrow$ Botón $\leftrightarrow$ GND
Buzzer	GPIO 22	Pin 29	GPIO $\rightarrow$ Buzzer $\rightarrow$ GND

3.3. Diagrama Esquemático del Circuito

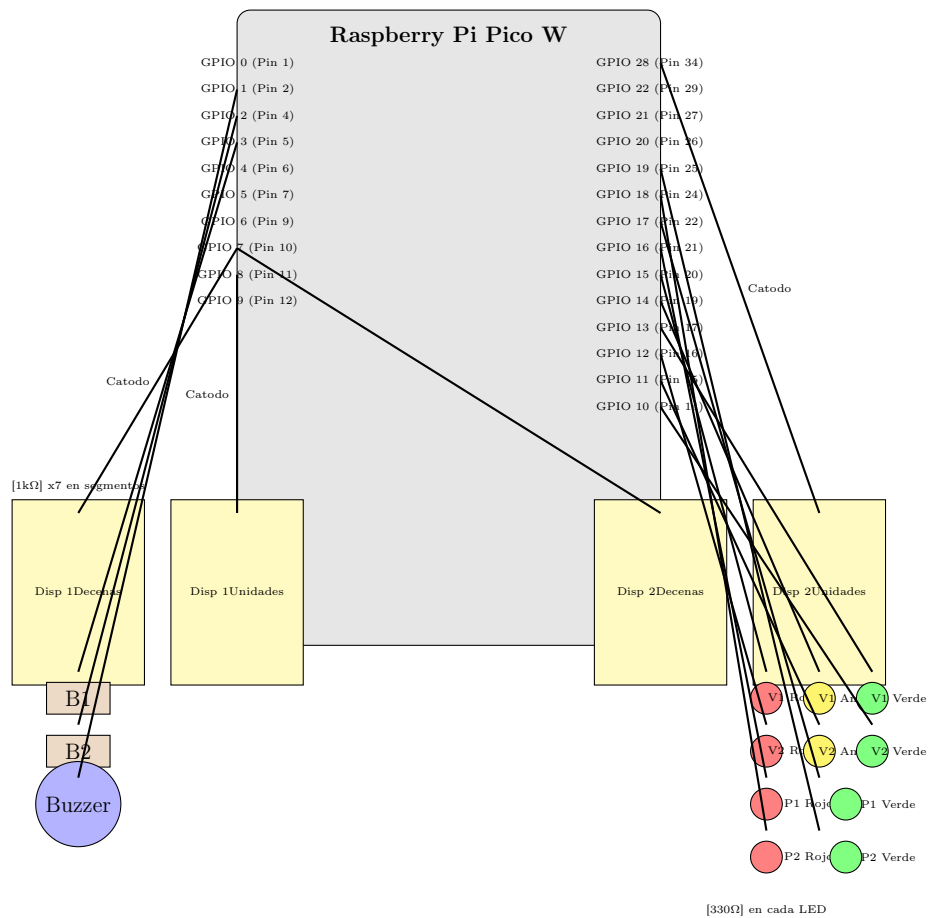
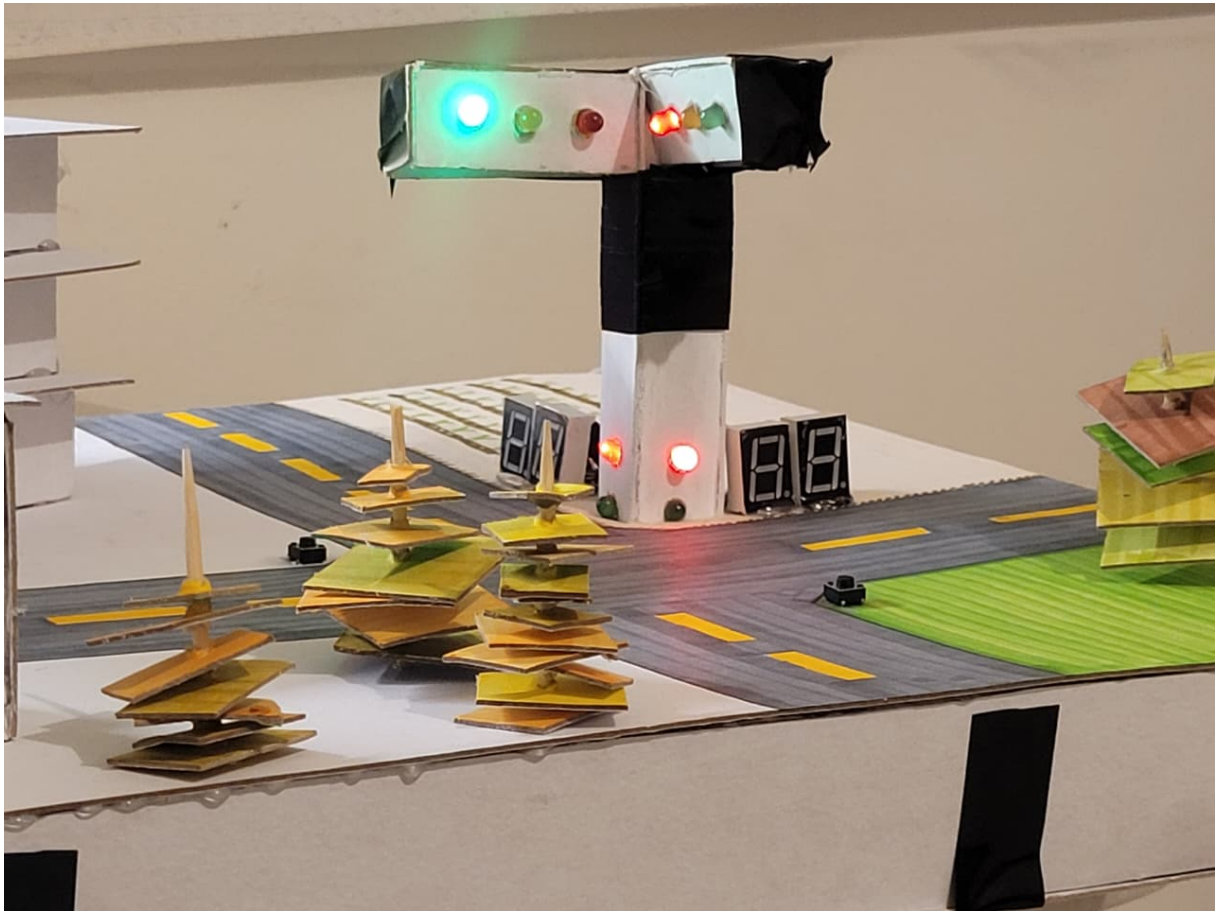


Diagrama Esquemático: Sistema de Cruce Peatonal

Figura 1: Diagrama esquemático completo del sistema de cruce peatonal con Raspberry Pi Pico W

3.4. Maqueta Física



3.5. Diagrama de Flujo de la Máquina de Estados

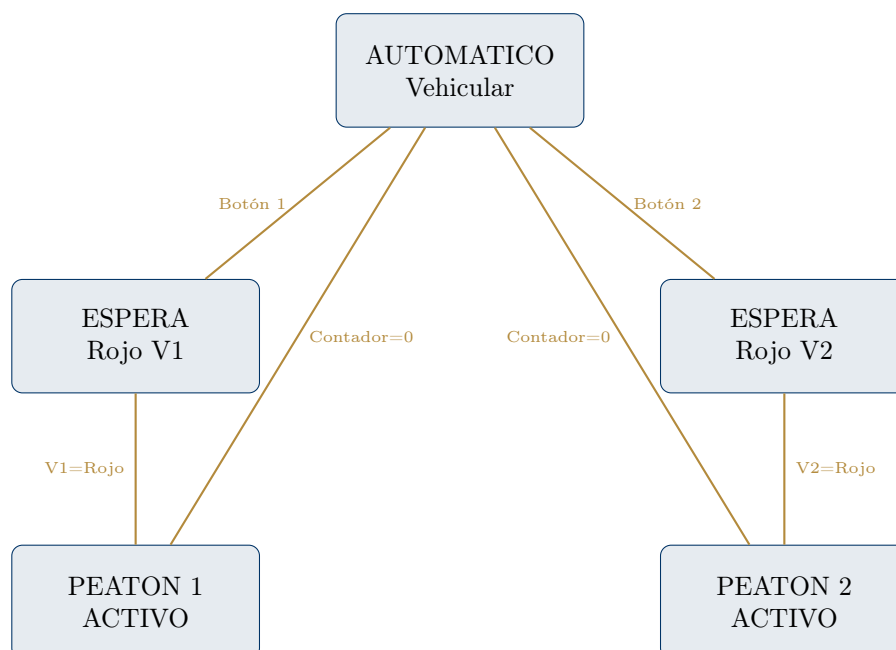


Figura 2: Diagrama de flujo de la máquina de estados

4. Código Fuente

4.1. Archivo: semaforo.h

```
1  #ifndef SEMAFORO_H
2  #define SEMAFORO_H
3
4  #include "pico/stdlib.h"
5  #include "hardware/timer.h"
6  #include "hardware/pwm.h"
7  #include "hardware/irq.h"
8
9  // Pines para displays 7 segmentos
10 #define SEG_A 0
11 #define SEG_B 1
12 #define SEG_C 2
13 #define SEG_D 3
14 #define SEG_E 4
15 #define SEG_F 5
16 #define SEG_G 6
17
18 // Pines para seleccion de displays (catodos)
19 #define DISP1_DEC 8
20 #define DISP1_UNI 9
21 #define DISP2_DEC 7
22 #define DISP2_UNI 28
23
24 // Pines semaforos vehiculares
25 #define V1_RED 15
26 #define V1_YELLOW 14
27 #define V1_GREEN 13
28 #define V2_RED 12
29 #define V2_YELLOW 11
30 #define V2_GREEN 10
31
32 // Pines semaforos peatonales
33 #define P1_RED 16
34 #define P1_GREEN 17
35 #define P2_RED 18
36 #define P2_GREEN 19
37
38 // Pines botones y buzzer
39 #define BUTTON1 20
40 #define BUTTON2 21
41 #define BUZZER 22
42
43 // Tiempos en segundos
44 #define TIME_GREEN 3
45 #define TIME_YELLOW 2
46 #define TIME_RED 5
47 #define TIME_PEDESTRIAN 10
48
49 // Estados del sistema
50 typedef enum {
51     STATE_VEHICULAR_AUTO,
52     STATE_WAIT_V1_RED,
53     STATE_P1_ACTIVE,
```



```
54     STATE_WAIT_V2_RED ,
55     STATE_P2_ACTIVE
56 } system_state_t;
57
58 typedef enum {
59     VEH_STATE_GREEN ,
60     VEH_STATE_YELLOW ,
61     VEH_STATE_RED
62 } vehicle_state_t;
63
64 // Prototipos de funciones
65 void init_hardware(void);
66 void update_traffic_lights(void);
67 void update_pedestrian_lights(void);
68 void update_displays(void);
69 void process_state_machine(void);
70 void buzzer_beep(int duration_ms, int times);
71 void multiplex_display(void);
72
73 #endif
```

Listing 1: Archivo de cabecera semaforo.h

4.2. Archivo: Practica_14.c (Principal)

```
1  #include "semaforo.h"
2  #include <stdio.h>
3
4  bool timer_callback(struct repeating_timer *t) {
5      process_state_machine();
6      update_traffic_lights();
7      update_pedestrian_lights();
8      update_displays();
9      return true;
10 }
11
12 bool fast_timer_callback(struct repeating_timer *t) {
13     multiplex_display();
14     return true;
15 }
16
17 int main() {
18     stdio_init_all();
19     sleep_ms(2000);
20
21     printf("Practica 14: Cruce Peatonal\n");
22     init_hardware();
23
24     struct repeating_timer timer;
25     add_repeating_timer_ms(1000, timer_callback, NULL, &timer);
26
27     struct repeating_timer fast_timer;
28     add_repeating_timer_ms(5, fast_timer_callback, NULL, &fast_timer)
29     ;
30
31     printf("Sistema iniciado\n");
```

```
31
32     while (1) {
33         tight_loop_contents();
34     }
35     return 0;
36 }
```

Listing 2: Programa principal

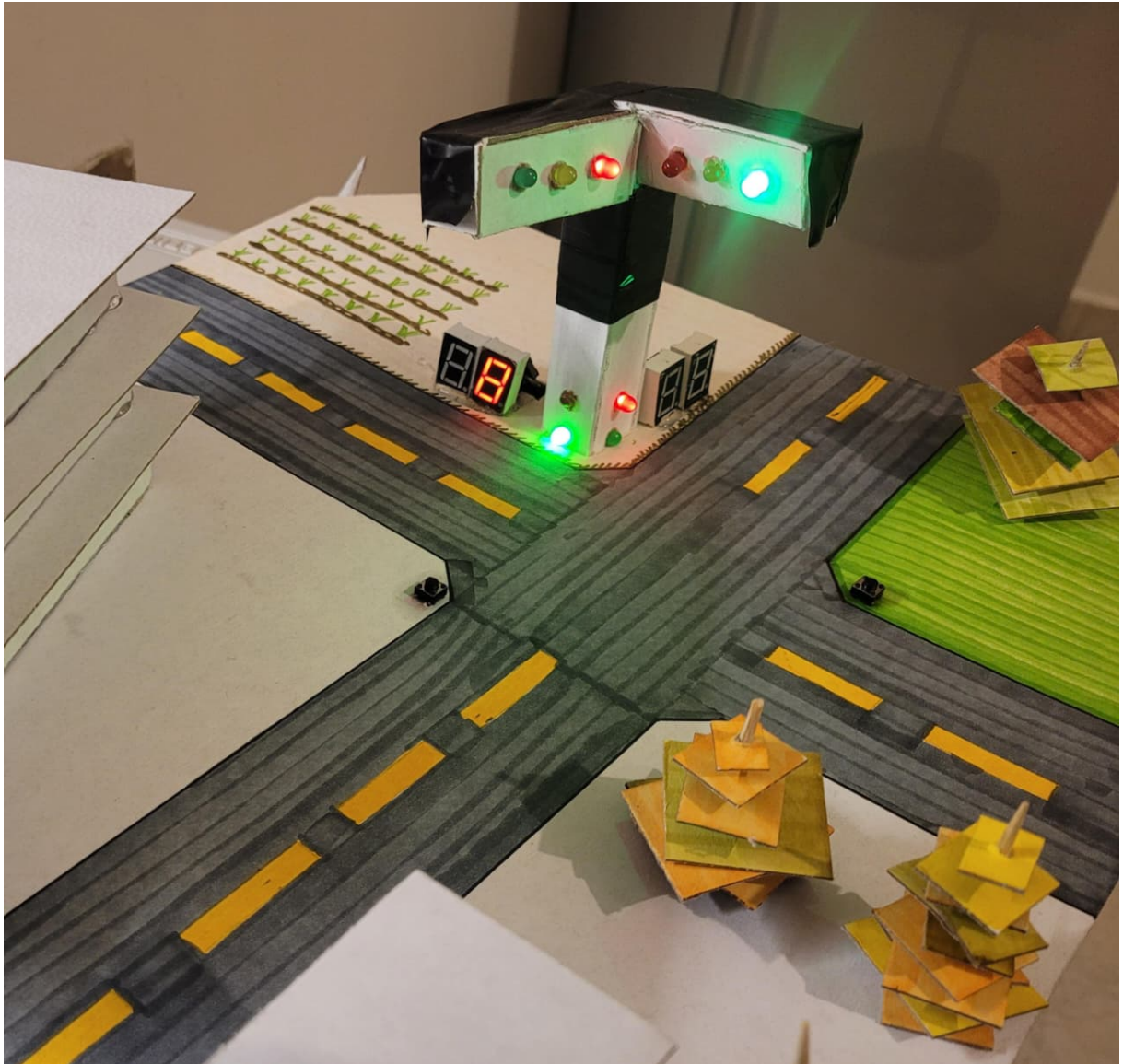
5. Resultados y Pruebas

5.1. Funcionamiento del Sistema

El sistema implementado cumple con todas las especificaciones solicitadas:

1. **Ciclo Automático:** Los semáforos vehiculares operan en ciclo continuo: Verde (3s) → Amarillo (2s) → Rojo (5s).
2. **Solicitud de Cruce Peatonal:** Al presionar un botón, el semáforo vehicular correspondiente completa su ciclo hasta rojo, luego activa el cruce peatonal con cuenta regresiva de 10 segundos.
3. **Sincronización:** Ambos semáforos vehiculares nunca muestran verde o amarillo simultáneamente.

5.2. Prueba de Funcionamiento



5.3. Dificultades y Soluciones

Tabla 5: Dificultades y soluciones

Dificultad	Solución
Botones no detectados	Implementación de polling con debounce por software
Sistema se bloqueaba al presionar botón	Reestructuración de la máquina de estados para actualizar timers en todos los estados
Displays no mostraban correctamente	Corrección de la lógica de multiplexación
Conflicto de pines GPIO	Reorganización de la asignación de pines

6. Conclusiones

La implementación del sistema de cruce peatonal con la Raspberry Pi Pico W ha permitido comprender y aplicar conceptos fundamentales de sistemas embebidos:

- **Máquinas de Estados:** La estructuración del código mediante una FSM resultó esencial para gestionar la lógica compleja del sistema de manera ordenada y libre de bloqueos.
- **Multiplexación:** La técnica de multiplexación permitió controlar 4 displays utilizando solo 11 pines GPIO.
- **Control de Tiempo Real:** El uso de timers periódicos del SDK de la Pico permitió una temporización precisa sin retardos bloqueantes.

6.1. Posibles Mejoras

- Aprovechar el módulo Wi-Fi para sincronizar múltiples cruces
- Incorporar sensores de presencia vehicular
- Implementar modo nocturno de bajo consumo
- Agregar pantalla LCD para mensajes informativos

7. Referencias

- [1] Raspberry Pi Foundation. (2024). *Raspberry Pi Pico W Datasheet*.
- [2] Raspberry Pi Foundation. (2024). *Pico SDK Documentation*.
- [3] ARM Limited. (2020). *Cortex-M0+ Technical Reference Manual*.